Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251221**
Nama Lengkap	**RIZKY FEBRIANTO KRISTIANSYAH**
Minggu ke / Materi	**10 / Manajemen File**

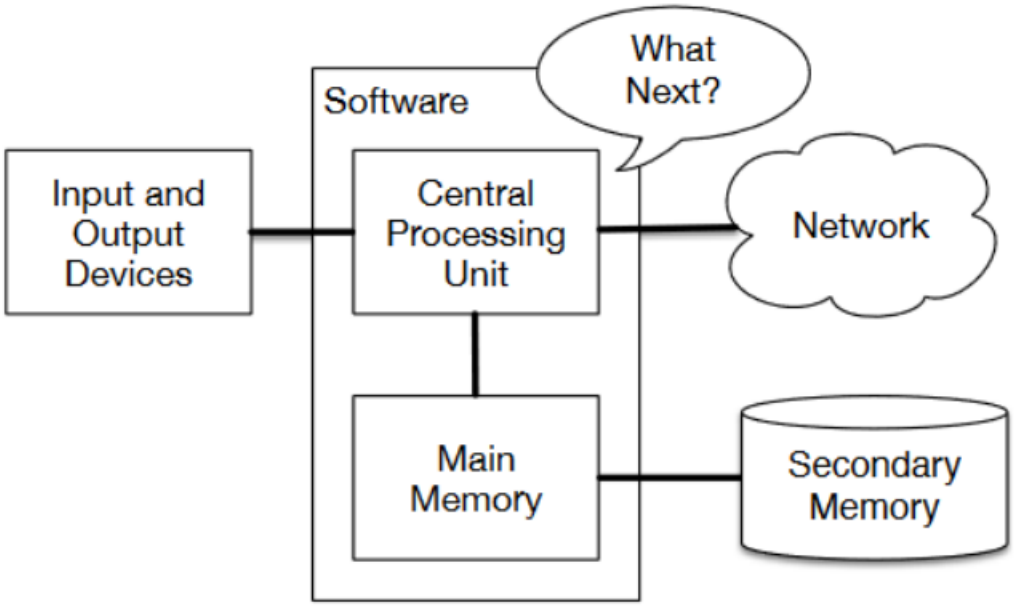
PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: Materi

Pengantar File

Program yang berjalan membutuhkan primary memory di dalam komputer. Semua data yang ada di dalam program tersebut disimpan ke dalam memory dan ketika program selesai dijalankan dan dimatikan, semua data di dalam program tersebut ikut hilang. Penyimpanan data di dalam memory bersifat tidak permanen (volatile). Oleh karena sifat tersebut, program yang menggunakan primary memory tidak dapat menyimpan data setelah program itu dimatikan.

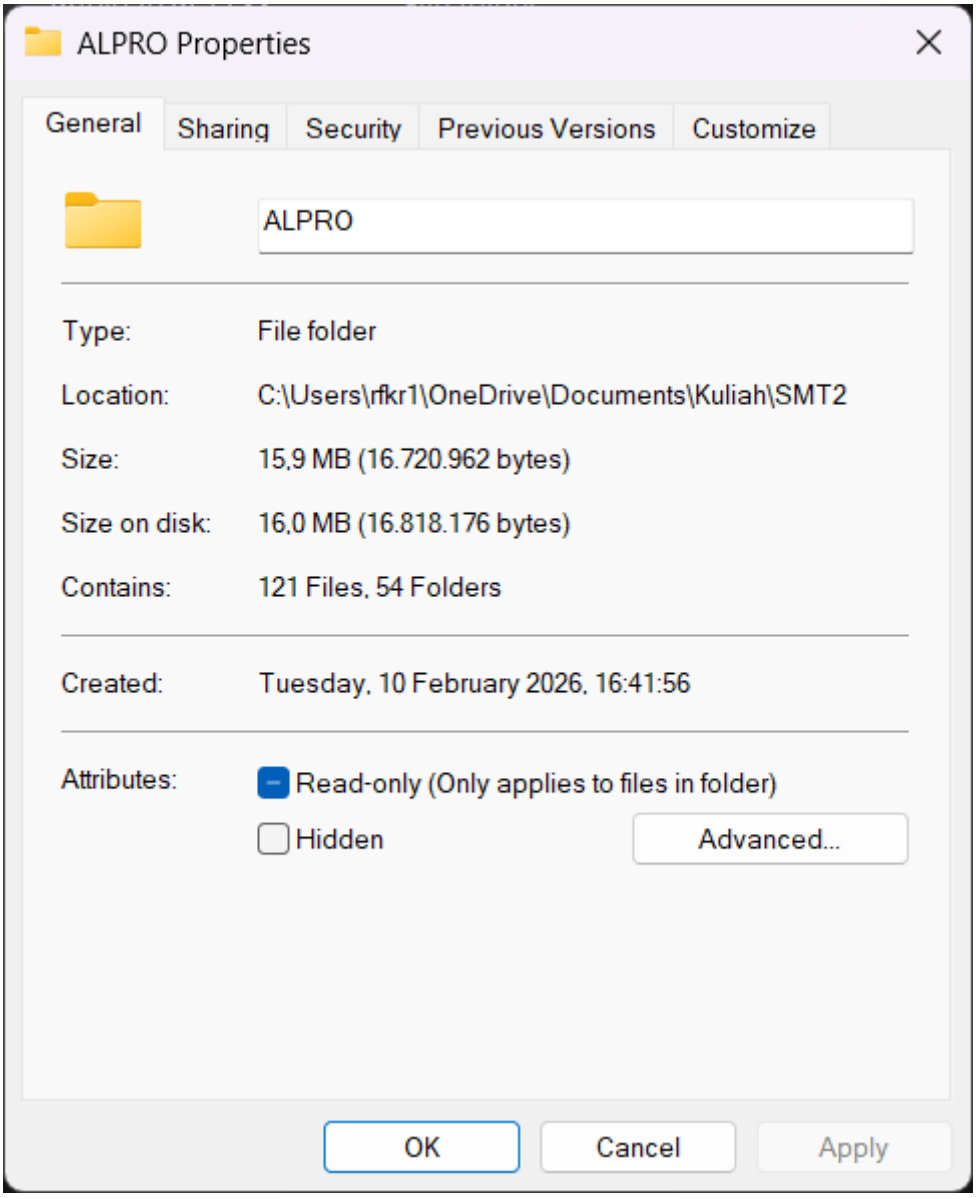
Untuk bisa menyimpan data pada program, diperlukan penyimpanan tetap yang dinamakan sebagai secondary memory. Konsep secondary memory dapat dilihat pada Gambar 1.1.



Gambar 1.1: Konsep Secondary Memory di Komputer

File disimpan pada secondary memory sehingga file dapat digunakan untuk menyimpan data dari program dan tidak akan hilang walaupun komputer dimatikan (Non-volatile). File pada dasarnya adalah bit-bit data yang disimpan di dalam secondary memory secara permanen, berupa kumpulan informasi yang saling berelasi satu sama lain sebagai satu kesatuan. File bisa berupa file system, ifle program (binary), file multimedia, file teks, dan lain sebagainya. File

memiliki property seperti nama file, ukuran, letak di harddisk, owner, hak akses, tanggal akses, dan lain-lain. Contoh property sebuah file dapat dillihat dari Gambar 1.2.



Gambar 1.2: Contoh File Properties

Pengaksesan File

Untuk mengakses file, langkah-langkah yang harus dilakukan adalah:

1. Menyiapkan file dan path yang akan diakses
2. Open file

3. Lakukan sesuatu dengan file tersebut, seperti ditampilkan (read) isinya atau diubah (write)
4. Close file

Program Python untuk mengakses file dapat dibuat sebagai berikut:

```
handle =  open('mbox-short.txt')
print(handle)
```

Hasilnya adalah:

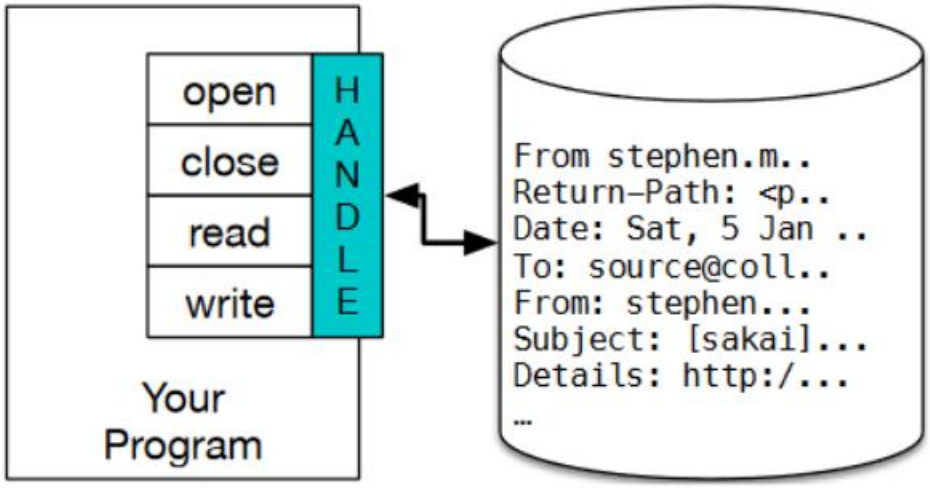
```
<_io.TextIOWrapper name='mbox-short.txt' mode='r' encoding='UTF-8'>
```

Jadi, hasilnya berupa tampilan nama file, modenya (r = read), dan encoding yang digunakan, yaitu unicode UTF-8 dari sistem io pada Python. Jika nama file tidak ada / tidak ditemukan, maka output akan error:

```
Traceback (most recent call last):  File "main.py", line 1, in <module>
handle =  open('tidak-ada.txt')
FileNotFoundError: [Errno 2] No such file or directory: 'tidak-ada.txt'
```

Pada file teks, biasanya file akan terdiri dari baris demi baris string. Cara pembacaan file teks biasanya juga menggunakan model baca baris demi baris untuk setiap string yang ditemukan sampai dengan EOF (End of File).

Ilustrasi handle file dapat dilihat pada Gambar 1.3:



Gambar 1.3: Ilustrasi Handle File

Manipulasi File

Untuk bisa memanipulasi file maka harus dimulai dari membaca file tersebut lebih dahulu. Cara untuk membaca file pada Python adalah:

1. Siapkan file
2. Open file
3. Loop setiap baris pada file
4. Close file

Cara pembacaan file dan menghitung jumlah baris dalam code editor seperti ini:

```
handle =  open('mbox-short.txt')
count = 0
for line in handle:
    count = count + 1
print('Line Count:', count)
```

Hasilnya adalah “Line Count : 1910”

Mengapa harus dibaca baris-perbaris? Untuk mengatasi kemungkinan sistem membuka file yang besar sekali dalam satu waktu dan akhirnya hang / crash karena ukuran file yang sangat besar.

Cara menampilkan file text dalam bytes, dapat menggunakan fungsi len dari string yang ada pada file:

```
handle =  open('mbox-short.txt')
hasil = handle.read()
print("Ukuran: " + len(hasil) + "bytes")
print("Huruf dari belakang sendiri mundur 16 huruf adalah: " + hasil[-16::1])
```

Program di atas akan open file mbox-short.txt, kemudian menampilkan ukuran berapa banyak huruf yang ada pada file tersebut (dengan catatan, kalau 1 karakter dianggap 1 byte, maka bisa disebut juga ukuran berapa banyak karakter = ukuran file tersebut dalam byte), dan terakhir menampilkan string dari huruf paling belakang maju 16 huruf ke depan.

Hati-hati! Perintah read() pada file sangat boros memory, sehingga jika ukuran file begitu besar maka lebih baik tidak menggunakan read(), melainkan menggunakan teknik loop seperti pada contoh di atas sebelumnya.

Selama dilakukan looping, kita juga dapat memanipulasi file tersebut, misalnya menampilkan bagian dari string. Seperti pada contoh file mbox, saat looping kita dapat hanya menampilkan kalimat yang diawali dengan “tanggal” saja, dengan source code sebagai berikut:

```
handle = open('mbox-short.txt')
count = 1
for line in handle:
    if line.startswith("Date:") and count <= 10:
        count += 1
        print(line)
```

Program di atas akan menghasilkan output 10 baris dengan string yang diawali dengan "Date: ", seperti ini:

```
Date: Sat, 5 Jan 2008 09:12:18 -0500

Date: 2008-01-05 09:12:07 -0500 (Sat, 05 Jan 2008)

Date: Fri, 4 Jan 2008 18:08:57 -0500

Date: 2008-01-04 18:08:50 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 16:09:02 -0500

Date: 2008-01-04 16:09:01 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 15:44:40 -0500

Date: 2008-01-04 15:44:39 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 15:01:38 -0500

Date: 2008-01-04 15:01:37 -0500 (Fri, 04 Jan 2008)
```

Mengapa ada baris kosong di setiap baris di atas? Karena dari file mbox sudah ada \n di setiap baris dan ditambah perintah print dari source code, sehingga terjadi double \n. Untuk mengatasinya, bisa menggunakan syntax rstrip() atau print tanpa \n

Penyimpanan File

Dalam Python, cara untuk menulis ke file sama dengan cara membuka (open) file, hanya saja metodenya yang diubah dari r (read) menjadi w (write). Untuk menuliskan isi string ke dalam file, langsung saja gunakan perintah handle.write(<string>) dan jangan lupa untuk tutup file dengan handle.close()

```
handle = open('output.txt','w')
tulisan = "teks ini akan dituliskan ke file\n"
handle.write(tulisan)
handle.close()
```

BAGIAN 2: LATIHAN MANDIRI

Link Github repository: https://github.com/ifgodwillsit/71251221_febrian

Latihan 10.1

```python
teksA = open("katanyaA.txt").readlines()
teksB = open("yangIniB.txt").readlines()
c = []
for i, (a, b) in enumerate(zip(teksA, teksB), start=1):
    if a != b:
        c.append(i)
        print(f"Baris {i}:")
        print(f"  [A] {a.strip()}")
        print(f"  [B] {b.strip()}")
if c != []:
    print(f"Jadi ada perbedaan di baris {c}")
else:
    print("Tidak ada perbedaan!")
```

Gambar 2.1: Sourcecode Latihan 10.1

```
katanyaA.txt
Beliau ini agaknya cukup keren
Cuma minusnya blio plenger dikit
GK kaya si itu yg josjis
Mari minta tutor josjis
Dari si paling josjis
```

Gambar 2.2: Teks pertama Latihan 10.1

```
yangIniB.txt
Beliau ini agaknya cukup keren
cuma minusnya beliau plenger dikit
sama kaya si itu yg josjis
Mari minta tutor josjis
Dari si paling plenger
```

Gambar 2.3: Teks kedua Latihan 10.1

Konteks dari Latihan 10.1 adalah membandingkan setiap baris dari 2 buah file, apakah ada perbedaan atau tidak, dan jika ada maka akan ditampilkan. Jadi konsep kodenya adalah pertama-tama membuka kedua file, karena menggunakan .readlines maka sistem membaca seluruh isi file sebagai list of strings. Setiap baris dalam file mewakili satu iterasi. Ada variabel c untuk menyimpan nomor baris yang berbeda. Fungsi zip() memasangkan baris-baris dari kedua file secara berurutan menjadi satu iterasi, sementara enumerate() memberikan nomor urut

baris dimulai dari 1. Di setiap iterasi (looping), baris dari file A dan B dicompare, apabila ada perbedaan maka nomor barisnya (i) disimpan di c dan print isi baris ke-i dari kedua file. Kemudian setelah loop selesai, jika c masih kosong maka akan menampilkan “Tidak ada perbedaan!”, sedangkan kalau c ada isinya maka dia akan menampilkan baris berapa saja yang berbeda antar kedua file.

```
Baris 2:
  [A] Cuma minusnya blio plenger dikit
  [B] cuma minusnya beliau plenger dikit
Baris 3:
  [A] GK kaya si itu yg josjis
  [B] sama kaya si itu yg josjis
Baris 5:
  [A] Dari si paling josjis
  [B] Dari si paling plenger
Jadi ada perbedaan di baris [2, 3, 5]
```

Gambar 2.4: Output Latihan 10.1

Latihan 10.2

```python
nama_file = "soal.txt"
print(f"nama file: {nama_file}\n")

for baris in open(nama_file).readlines():
    soal, jawaban_benar = baris.strip().split(" || ")
    print(soal)
    jawaban = input("Jawab: ")
    if jawaban.lower() == jawaban_benar.lower():
        print("Jawaban benar!")
    else:
        print("Jawaban salah!")
```

Gambar 2.5: Sourcecode Latihan 10.2

```
soal.txt
1+1 = || 2
Bendera Indonesia? || Merah Putih
Kota gudeg adalah: || Yogyakarta
Komponen PC untuk penyimpanan file adalah... || harddisk
50 * 20 = || 1000
```

Gambar 2.6: Teks Latihan 10.2

Konteks soal ini adalah diberikan soal dan kunci jawaban di teks soal.txt. Kemudian user perlu menjawab atau memberi input, jika sesuai dengan kunci jawaban (TIDAK CASE SENSITIVE) maka akan print “Jawaban benar!” Jika tidak sesuai, maka print “Jawaban salah!”. Jadi pada sourcecode pertama-tama membuka file, lalu melooping tiap baris dari file untuk memisahkan

antara soal dan kunci jawaban. Soal didisplay kemudian user diminta memasukkan jawaban. Jika setelah dicocokkan antara input user dengan jawaban benar memberikan nilai False, maka jawabannya salah. Jika True, jawabannya benar. Perlu diingat karena dari soalnya jawaban tidak perlu case sensitive maka user input dengan kunci jawaban dapat diseragamkan dengan dilowercase.

```
nama file: soal.txt

1+1 =
Jawab: 2
Jawaban benar!
Bendera Indonesia?
Jawab: Merah putih
Jawaban benar!
Kota gudeg adalah:
Jawab: yogyakarta
Jawaban benar!
Komponen PC untuk penyimpanan file adalah...
Jawab: HARDDISK
Jawaban benar!
50 * 20 =
Jawab: 1000
Jawaban benar!
```

Gambar 2.7: Output Latihan 10.2